

Documentation as Project Memory in AI-Assisted Development

A practical case study in keeping AI-generated software bounded, reviewable, and maintainable.

Summary

AI-assisted development has a context problem. Coding agents can implement quickly, but they depend heavily on the context they receive. A long planning conversation is often poor implementation context: it may contain rejected options, future ideas, shorthand, loose assumptions, and half-decisions.

For a human, that is normal design exploration. For a coding agent, it can become accidental permission.

This case study describes a workflow for reducing that risk by treating documentation as project memory. The implementation was tested under a strict constraint: every code change in the project went through the AI-assisted workflow. I did not write code directly, even when it would have been faster.

The core pattern is:

Explore in conversation. Build from a contract. Preserve the result in documentation. Reuse that documentation as context for the next task.

The case study is intended to be read alongside the project documents and three consecutive Codex Contracts:

- DEV_ARCHITECTURE.md
- DEV_DATA_SHAPES.md
- DEV_API_CONTRACTS.md
- TECHNICAL_DEBT.md
- CP-0011.txt
- CP-0012.txt
- CP-0013.txt

Together, these documents show the method operating across a short development sequence rather than a single isolated task.

Problem: context decay

The practical risk in AI-assisted development is context decay: decisions are made faster than they are preserved.

Architecture, API behavior, data shapes, validation rules, temporary compromises, and deferred work can disappear into chat history or human memory. Once that happens, later tasks start from incomplete or stale context.

The solution in this implementation was to make documentation part of the development transaction.

Documentation is not written “after the work.” It is a required output of the work.

If a task changes architecture, the architecture document changes. If it changes an API response, the API contract changes. If it changes stored data, the data shape document changes. If it exposes a problem that should not be fixed yet, the technical debt document changes.

In this project, the main memory files are:

- DEV_ARCHITECTURE.md
- DEV_DATA_SHAPES.md
- DEV_API_CONTRACTS.md
- TECHNICAL_DEBT.md

These files are operational project memory. They preserve decisions, constrain future work, and provide current context for later AI-assisted tasks.

Implementation: TB / CC / FR

The workflow uses three stages:

- **Task Brief:** explore the problem.
- **Codex Contract:** define the bounded implementation.
- **Final Review:** check the result and update project memory.

The control model is two-way review: the human reviews the AI's output, and the assistant reviews the human's instructions for ambiguity, scope creep, and loose language. The human decides when the contract is clear enough to hand off, and when the result is acceptable.

In practice:

explore → contract → review → implement → test → review result → patch code or docs → continue from updated documentation.

Task Brief

The Task Brief is where the task is explored before it becomes implementation work.

The goal is to identify scope, risks, constraints, non-goals, affected files, expected behavior, and documentation impact.

This stage may include uncertainty, alternative designs, and questions. It is not sent directly to the coding agent.

The assistant is useful here as a reviewer and translator. It helps convert engineering instincts into clearer technical language:

- “This feels unsafe” becomes “this crosses a trust boundary.”
- “Do not save failed AI output” becomes “protect the validation and commit boundary.”

- “This ugly thing should not be fixed now” becomes “record it as technical debt and keep it out of scope.”

The output of the Task Brief is not code. The output is a clearer implementation target.

Codex Contract

The Codex Contract is the bounded instruction set given to the coding agent.

It is not a transcript of the planning conversation. It is the cleaned-up implementation contract.

A good contract states:

- the goal;
- relevant context;
- files likely involved;
- behavior that must be preserved;
- non-goals;
- acceptance criteria;
- verification steps;
- required documentation updates;
- final report requirements.

The non-goals are critical. With coding agents, “do not do this” is often as important as “do this.” If the contract does not explicitly say not to refactor a legacy route, the agent may refactor it. The refactor may be technically reasonable and still be wrong for the current task.

The contract defines the box. Documentation updates are part of that box.

Final Review

Final Review checks both implementation and memory.

The review asks:

- Did the implementation match the contract?
- Did it preserve required behavior?
- Did it avoid unrelated changes?
- Did it update the correct documentation?
- Did it record deferred work as technical debt?
- Did the report accurately describe what changed?
- Does anything need a bounded patch?

Manual testing remains required. The assistant can help review the coding agent’s return, but it does not replace human acceptance.

The workflow is therefore not:

Write code, then someday update docs.

It is:

Design the slice, implement the slice, update the docs, review the result, patch if needed, repeat.

That is the mechanism that turns documentation into project memory.

Why contract review matters

The contract review step exists because conversational language can leak into technical instructions.

A simple example from this case study is the phrase “dynamic frontend.” In conversation, that is understandable shorthand. The project has a browser interface with dynamic behavior: polling, player input submission, scene rendering, choices, Chronicle rendering, and rollback wiring.

But the project-memory wording is more precise:

a static browser frontend served by the Node backend, with dynamic client-side behavior owned by `public/app.js`.

That distinction matters. In a CC, “dynamic frontend” could be misread as permission to introduce a frontend framework, server-side rendering, a build step, or a broader UI restructure. None of that is intended.

This is exactly the failure mode the workflow is designed to catch. Human shorthand is useful during exploration. It must be cleaned up before implementation.

Representative sequence: CP-0011 to CP-0013

`CP-0011`, `CP-0012`, and `CP-0013` are included as three consecutive Codex Contracts from the same development phase.

They are useful together because they show the workflow being applied repeatedly across different kinds of context work:

- `CP-0011` establishes a narrow Thread Context Controller v0.
- `CP-0012` establishes a Messages Side Channel v0.
- `CP-0013` deprecates legacy Enforcement data and adds clean scoped Enforcement v0.

The details differ, but the contract pattern is the same. Each task defines a bounded implementation, preserves legacy behavior, names what must not be changed, gives acceptance criteria, requires verification, and tells Codex what to report back.

`CP-0013` is the strongest documentation-memory example because it explicitly requires narrow updates across:

- `DEV_DATA_SHAPES.md`
- `DEV_API_CONTRACTS.md`
- `DEV_ARCHITECTURE.md`
- `TECHNICAL_DEBT.md`

The sequence matters. It shows that documentation as project memory was not a one-off instruction. It was part of the operating pattern across consecutive development passes.

Example pattern

Assume a feature is hardcoded in a legacy file. It should eventually become a cleaner system, but the current task should only create a safe v0 seam.

A weak instruction would be:

Refactor this feature into a better system.

That instruction has no boundary.

A better Task Brief clarifies the scope:

We are not building the full system yet. We are creating a clean v0 seam. The old legacy data must remain. The new code should read clean records, skip legacy records, and expose a separate API field. No migration. No deletion. No broad refactor.

The Codex Contract turns that into implementation instructions:

Create a controller for clean v0 records. Preserve the old API field. Add a new clean projection. Do not rewrite existing files. Do not change JSON loading behavior. Do not refactor unrelated legacy routes. Update architecture notes, API contracts, data shapes, and technical debt only where this task changes them. Report changed files, validation rules, documentation updates, and risks.

Now the coding agent has a bounded task. It knows what to build, what not to touch, where documentation belongs, and what to report back.

Final Review then checks the implementation, the report, and the documentation updates. Any correction goes back through the loop as a bounded patch.

Why it works

This pattern works because documentation becomes part of the development transaction.

The documents are updated while the context is fresh. The coding agent receives a bounded contract instead of a messy conversation. The assistant can reason from current project documents instead of stale chat history. The developer does not have to keep the whole system in working memory.

The result is not perfect AI coding. The result is reviewable AI coding.

A change can be evaluated against the contract. A working implementation can still be rejected if it violates scope. Deferred work can be preserved without being accidentally implemented. Future tasks can begin from documented project state.

Scope

The implementation described here was built as a local prototype with:

- Node backend;
- static browser frontend with dynamic client-side behavior;
- local JSON persistence;
- rollback support;
- validation gates;
- LLM gateway compatible with local or OpenAI-style providers.

ChatGPT was used for planning, review, and learning. Codex was used as the coding agent. LM Studio was used for local model testing.

The exact tools are not the important part. The transferable pattern is **conversation for exploration, contract for implementation, documentation for memory**.

Under this implementation, documentation acted as project memory because three conditions were enforced:

1. documentation updates were required by the implementation contract;
2. documentation updates were checked during final review;
3. updated documentation was reused as context for later tasks.

The workflow does not remove the need for human judgment. It depends on it. The human still owns scope, acceptance, risk decisions, and final review.

Conclusion

Documentation as project memory is a practical control mechanism for AI-assisted development.

The value is not that documentation exists. The value is that documentation becomes operational.

It constrains the next task, preserves decisions, records deferred work, and reduces dependence on chat history or human memory.

That is what kept the project understandable under a prompt-only implementation constraint.